competing for a mutex, it *will* impact the performance significantly. Rather than trying to shave a small fraction of time off the mutex operations, it would probably be more beneficial to restructure the application so that there's less contention on the mutex. Designing applications to reduce contention is covered in chapter 8.

In those very rare cases where the C++ Standard Library does not provide the performance or behavior required, it might be necessary to use platform-specific facilities.

1.3.4. Platform-specific facilities

Although the C++ Thread Library provides reasonably comprehensive facilities for multithreading and concurrency, on any given platform there will be platform-specific facilities that go beyond what's offered. In order to gain easy access to those facilities without giving up the benefits of using the Standard C++ Thread Library, the types in the C++ Thread Library may offer a `native_handle()` member function that allows the underlying implementation to be directly manipulated using a platform-specific API. By its very nature, any operations performed using the `native_handle()` are entirely platform dependent and out of the scope of this book (and the Standard C++ Library itself).

Of course, before even considering using platform-specific facilities, it's important to understand what the Standard Library provides, so let's get started with an example.

1.4. Getting started

OK, so you have a nice, shiny C++11-compatible compiler. What next? What does a multithreaded C++ program look like? It looks pretty much like any other C++ program, with the usual mix of variables, classes, and functions. The only real distinction is that some functions might be running concurrently, so you need to ensure that shared data is safe for concurrent access, as described in chapter 3. Of course, in order to run functions